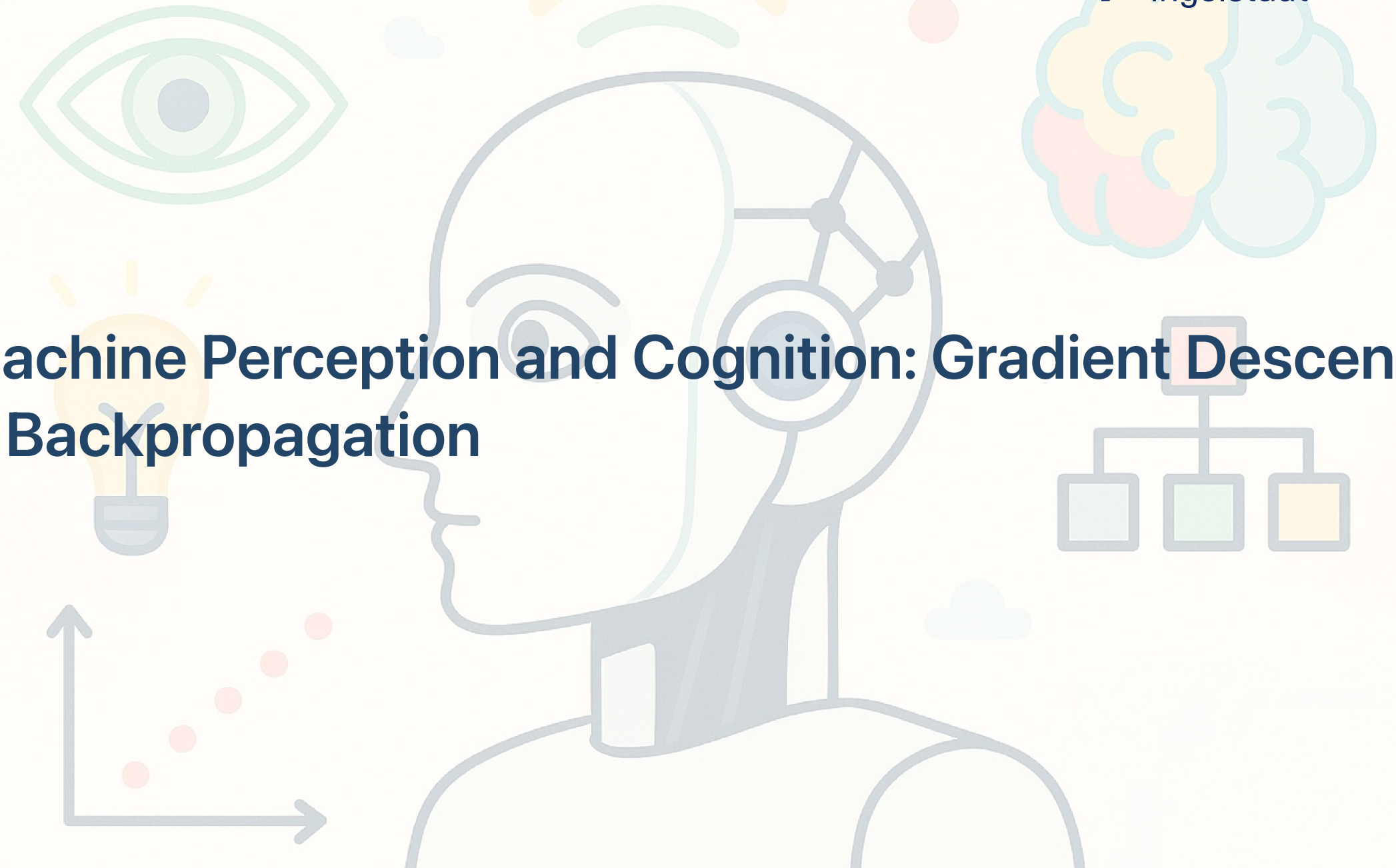
Technische Hochschule Ingolstadt



Machine Perception and Cognition: Gradient Descent & Backpropagation

Learning from data

The MLP and CNN architectures define **how** information flows through the network. The remaining question is: **how do we find the right weights?**

Given a training dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^{N}$, we want to find weights $\mathbf{W}$ such that the network output $f(\mathbf{x}; \mathbf{W})$ is close to the target $\mathbf{y}$ for all training samples.

This requires:

1. A **loss function** $\mathcal{L}(\mathbf{W})$ that measures how bad the current predictions are
2. An **optimization algorithm** that iteratively improves $\mathbf{W}$
3. A **gradient computation** method that tells us how to update each weight

Empirical risk minimization

The **empirical risk** is the average loss over the training set:

$$\mathcal{L}(\mathbf{W}) = \frac{1}{N} \sum_{i=1}^{N} \ell\Big(f(\mathbf{x}^{(i)}; \mathbf{W}),\ \mathbf{y}^{(i)} \Big)$$

where $\ell(\hat{\mathbf{y}}, \mathbf{y})$ is the **per-sample loss** comparing prediction $\hat{\mathbf{y}}$ to target $\mathbf{y}$.

Training goal:

$$\mathbf{W}^* = \arg\min_{\mathbf{W}} \mathcal{L}(\mathbf{W})$$

A good loss function

A good loss function should be:

- **Non-negative** and **zero** only when predictions are perfect
- **Differentiable** so that we can compute gradients
- **Matched to the task**: regression vs. classification

Regression: Mean Squared Error (MSE)

For **regression** tasks, the network predicts a real-valued output $\hat{y} = f(\mathbf{x}; \mathbf{W})$.

The **Mean Squared Error (MSE)** loss averages the squared prediction error over N samples:

$$\mathcal{L}_{\text{MSE}}(\mathbf{W}) = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

For a single sample with scalar output:

$$\ell_{\text{MSE}}(\hat{y},\, y) = \left(\hat{y} - y \right)^2$$

The gradient with respect to the prediction is:

$$\frac{\partial \ell_{\text{MSE}}}{\partial \hat{y}} = 2(\hat{y} - y)$$

MSE loss — properties

$$\ell_{\text{MSE}}(\hat{y}, y) = (\hat{y} - y)^2$$

- ✅ Always **non-negative**; equals zero only when $\hat{y} = y$
- ✅ Penalizes **large errors** disproportionately (quadratic)
- ✅ **Differentiable** everywhere — gradient is well-defined
- ✅ Simple closed-form derivative: $\frac{\partial \ell}{\partial \hat{y}} = 2(\hat{y} - y)$
- ❌ Sensitive to **outliers** — a single large error dominates the loss
- 📐 MSE corresponds to **maximum likelihood estimation** under a Gaussian noise model

Classification: the softmax function

For **multi-class classification** with K classes, the last layer produces a raw score vector $\mathbf{z}^{(L)} \in \mathbb{R}^K$.

The **softmax** function converts raw scores into a valid **probability distribution**:

$$\hat{y}_k = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \quad k = 1, \ldots, K$$

Softmax function - properties

Properties:

- $\hat{y}_k > 0$ for all k — all class probabilities are positive
- $\sum_{k=1}^{K} \hat{y}_k = 1$ — valid probability distribution
- Preserves the **ranking** of raw scores
- Larger differences between scores produce a more **peaked** distribution

Softmax — effect of score differences

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

z_1	z_2	z_3	$\hat{y}_1$	$\hat{y}_2$	$\hat{y}_3$
1.0	2.0	3.0	0.09	0.24	0.67
0.1	0.2	0.3	0.30	0.33	0.37
10	20	30	≈ 0	≈ 0	≈ 1

➡️ Larger score differences produce **more confident** predictions (peaked distribution).

➡️ Equal scores produce a **uniform** distribution (maximum uncertainty).

Classification: cross-entropy loss

Given softmax output $\hat{\mathbf{y}}$ and a probabilistic target $\mathbf{y} \in \{0, 1\}^K$ (e.g., $\mathbf{y} = [0, 1, 0]^\top$), the **cross-entropy loss** is:

$$\ell_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$

In one-hot cases where only the true class c has $y_c = 1$, this simplifies to:

$$\ell_{\text{CE}} = -\log \hat{y}_c$$

The full dataset loss:

$$\mathcal{L}_{\text{CE}}(\mathbf{W}) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} y_k^{(i)} \log \hat{y}_k^{(i)}$$

Cross-entropy loss — properties

$$\ell_{\mathrm{CE}}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$

- ✅ **Non-negative**: since $0 < \hat{y}_k \leq 1$, we have $-\log \hat{y}_k \geq 0$
- ✅ **Zero** when $\hat{y}_c = 1$ for c being the true class in a one-hot encoding
- ✅ Grows **rapidly** as $\hat{y}_c \to 0$ (very wrong predictions are heavily penalized)
- ✅ **Differentiable** — combined with softmax the gradient has an elegant form
- ✅ **Directly optimizes** the probability assigned to the correct class
- 📐 Cross-entropy is the **negative log-likelihood** under a categorical distribution

The optimization problem

Training a neural network means finding weights $\mathbf{W}^*$ that minimize the loss:

$$\mathbf{W}^* = \arg\min_{\mathbf{W}} \mathcal{L}(\mathbf{W})$$

Challenges:

- $\mathcal{L}$ is a function of **millions** of parameters (e.g., a ResNet-50 has ~25 million)
- The loss surface is **non-convex** — many local minima and saddle points
- We cannot evaluate $\mathcal{L}$ analytically; only numerically on a data batch

Solution: **iterative optimization** using gradient information

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} \mathcal{L}(\mathbf{W})$$

Gradient of the loss

The **gradient** of $\mathcal{L}$ with respect to all weights is a vector (or matrix) of partial derivatives:

$$\nabla_{\mathbf{W}} \mathcal{L} = \left\{ \frac{\partial \mathcal{L}}{\partial W_{jk}^{(l)}} \right\}_{l,j,k}$$

Each component $\frac{\partial \mathcal{L}}{\partial W_{jk}^{(l)}}$ tells us:

- If **positive**: increasing $W_{jk}^{(l)}$ increases the loss
- If **negative**: increasing $W_{jk}^{(l)}$ decreases the loss

The gradient always points in the direction of **steepest ascent** of $\mathcal{L}$.

➡️ We move in the **negative gradient direction** to decrease the loss.

Gradient descent

Gradient descent iteratively updates all weights in the direction of the **negative gradient**:

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}$$

where $\eta > 0$ is the **learning rate** hyperparameter.

Repeat until convergence:

1. **Forward pass**: compute predictions $\hat{\mathbf{y}}$ and loss $\mathcal{L}$
2. **Backward pass**: compute gradients $\nabla_{\mathbf{W}} \mathcal{L}$ (via backpropagation)
3. **Weight update**: adjust all weights using the gradient

Each such cycle is called an **iteration** or **step**.

Gradient descent animation

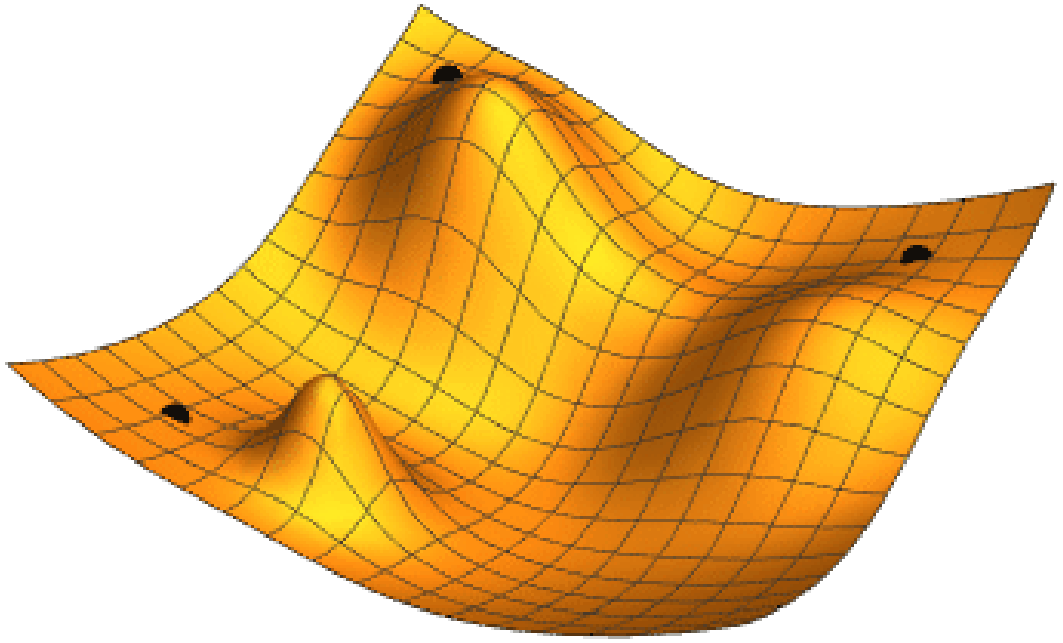


Image source: Jacopo Bertolotti, CC0, via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Gradient_descent.gif

The learning rate

The **learning rate** η controls how large each update step is.

	η **too small**	η **too large**	η **well-chosen**
Behavior	Very slow convergence	Oscillates or diverges	Steady convergence

Best practices:

- Typical starting values: $\eta \in [10^{-4}, 10^{-1}]$
- Use a **learning rate schedule**: reduce η over training (e.g., cosine annealing)
- Adaptive optimizers like **Adam** automatically scale the learning rate per parameter

Batch, stochastic, and mini-batch gradient descent

Computing the gradient over the full dataset is expensive. In practice, we use **subsets**:

Method	Gradient over	Update frequency	Cost per step
Batch GD	Full dataset (N)	Once per epoch	High
Stochastic GD (SGD)	Single sample	$N\times$ per epoch	Low
Mini-batch GD	Batch of B samples	$N/B\times$ per epoch	Medium

Mini-batch gradient descent

Mini-batch gradient descent is the standard in deep learning:

- Better gradient estimate than SGD
- Exploits GPU parallelism efficiently
- Typical batch sizes: $B \in \{32, 64, 128, 256\}$

One full pass over the training data = one **epoch**.

Mini-batch gradient descent

For a mini-batch $\mathcal{B} \subset \{1, \ldots, N\}$ of size B:

$$\mathcal{L}_{\mathcal{B}}(\mathbf{W}) = \frac{1}{B} \sum_{i \in \mathcal{B}} \ell\Big(f(\mathbf{x}^{(i)}; \mathbf{W}),\ \mathbf{y}^{(i)} \Big)$$

Update rule:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} \mathcal{L}_{\mathcal{B}}(\mathbf{W})$$

The mini-batch gradient is an **unbiased, noisy estimate** of the true gradient. This noise:

- ✅ Helps **escape shallow local minima** and saddle points
- ✅ Provides a form of **implicit regularization**
- ❌ Introduces variance — the update direction fluctuates between batches

SGD with momentum

Plain SGD can oscillate and converge slowly. **Momentum** accumulates a velocity vector to smooth the update direction:

$$\mathbf{v} \leftarrow \mu\,\mathbf{v} - \eta\,\nabla_{\mathbf{W}}\mathcal{L}$$
$$\mathbf{W} \leftarrow \mathbf{W} + \mathbf{v}$$

where $\mu \in [0, 1)$ is the **momentum coefficient** (typically $\mu = 0.9$).

Effect:

- Accumulates gradient in consistent directions → **faster progress** in flat regions
- Dampens oscillations in directions with high curvature
- Analogous to a ball rolling down a hill and picking up speed

Practical training

A full training loop for one epoch:

1. **Shuffle** the training data
2. For each mini-batch $\mathcal{B}$:
   a. **Forward pass**: compute $\hat{\mathbf{y}}$ and $\mathcal{L}_{\mathcal{B}}$
   b. **Backward pass**: compute $\nabla_{\mathbf{W}} \mathcal{L}_{\mathcal{B}}$ (backpropagation)
   c. **Update**: $\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} \mathcal{L}_{\mathcal{B}}$
3. Evaluate on **validation set** to monitor for overfitting
4. Adjust learning rate if using a schedule

Repeat for many epochs (typically tens to thousands) until convergence.

The loss landscape

The loss $\mathcal{L}(\mathbf{W})$ defines a high-dimensional surface over all weight configurations:

- For a 235k-parameter MLP: a surface in $\mathbb{R}^{235\,000}$!
- Contains **many local minima** and **saddle points**
- **Saddle points** are more common than local minima in high dimensions

➡️ Mini-batch noise, momentum, and adaptive optimizers all help navigate this landscape.

Local and global minima

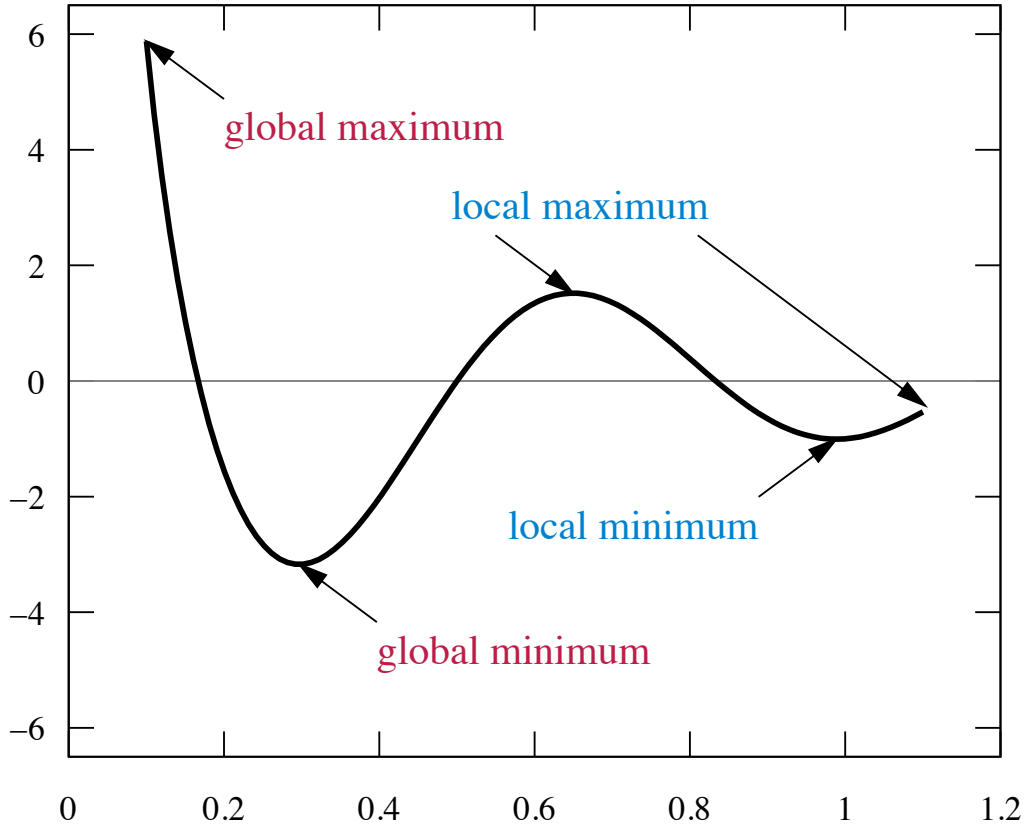


Image source: KSmrq, CC BY-SA 3.0 (https://creativecommons.org/licenses/by-sa/3.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Extrema_example.svg

Local minima and difficult loss surfaces

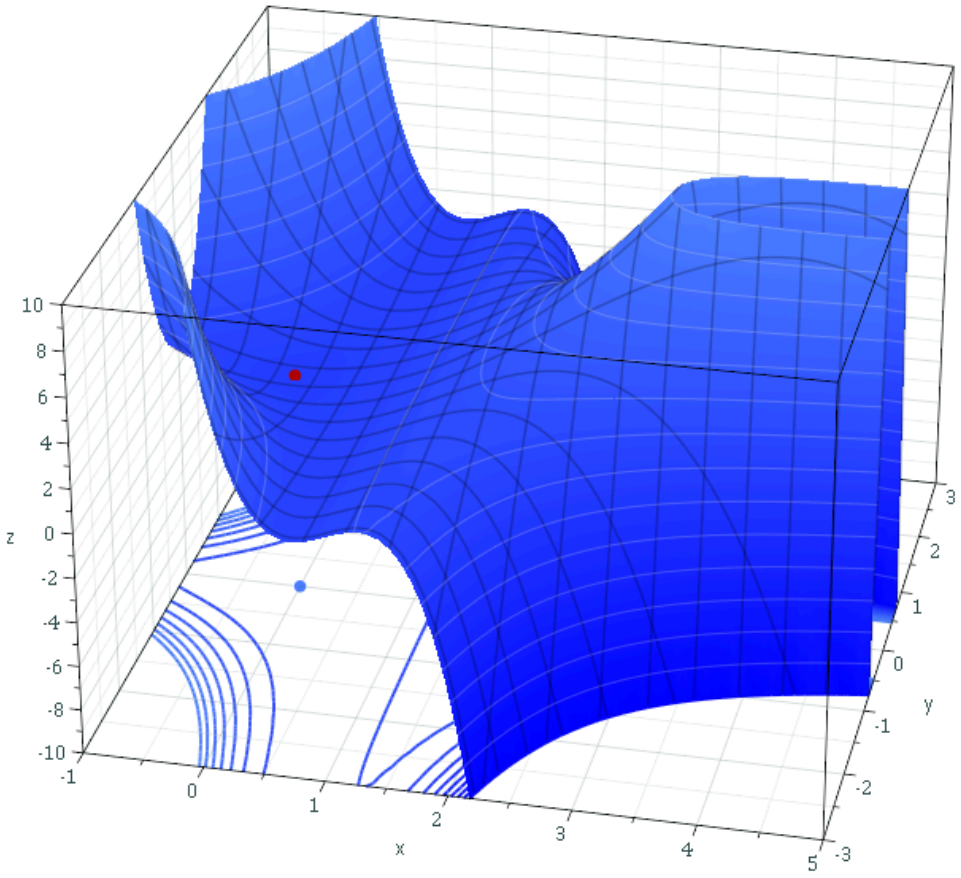


Image source: Sam Derbyshire, CC BY-SA 3.0 (https://creativecommons.org/licenses/by-sa/3.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:MaximumCounterexample.png

Saddle points

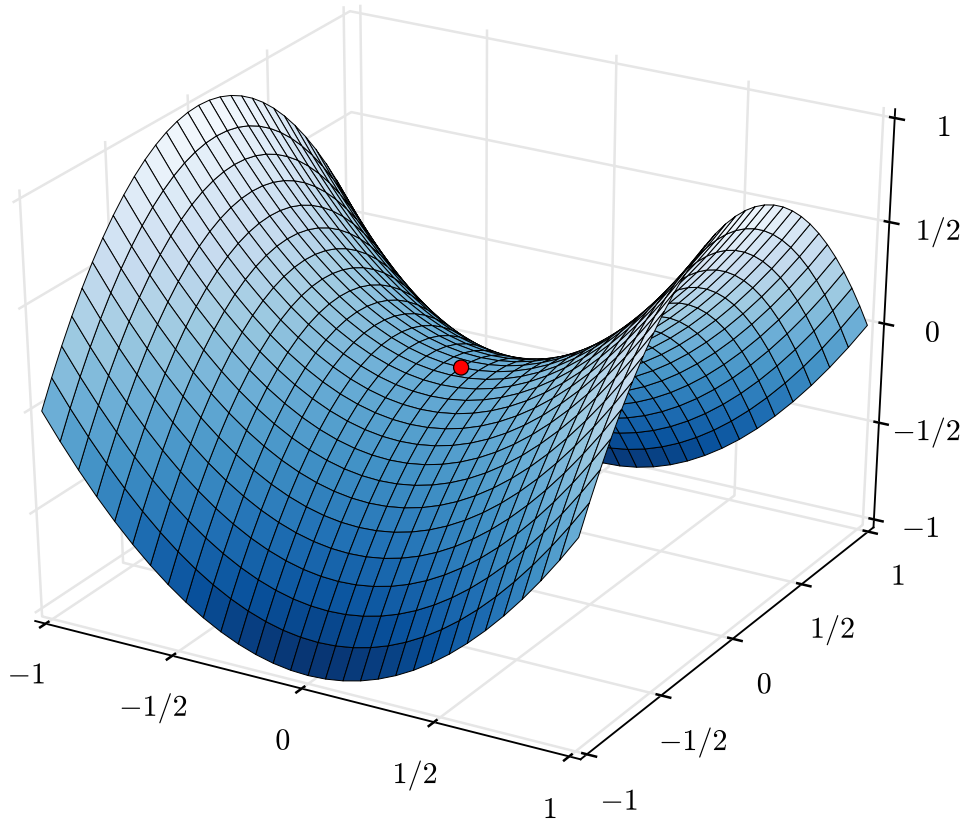


Image source: Nicoguaro, CC BY 3.0 (https://creativecommons.org/licenses/by-sa/3.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Saddle_point.svg

Backpropagation — the key question

Gradient descent requires computing $\nabla_{\mathbf{W}} \mathcal{L}$ — the partial derivative of the loss with respect to **every single weight** in the network.

For an L-layer MLP with P total parameters, this is P partial derivatives:

$$\frac{\partial \mathcal{L}}{\partial W_{jk}^{(1)}}, \quad \ldots, \quad \frac{\partial \mathcal{L}}{\partial W_{jk}^{(L)}}$$

Naïve finite differences: perturb each weight by ϵ and measure the loss change — requires $2P$ forward passes per update. For $P = 235\,000$: completely intractable.

Backpropagation: computes all P gradients in a single **backward pass** — same cost as one forward pass.

Chain rule

For composite functions $f(g(x))$, the chain rule gives:

$$\frac{df}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx}$$

For a chain $y = f_L(f_{L-1}(\cdots f_1(x)\cdots))$:

$$\frac{dy}{dx} = \frac{\partial f_L}{\partial f_{L-1}} \cdot \frac{\partial f_{L-1}}{\partial f_{L-2}} \cdots \frac{\partial f_1}{\partial x}$$

Chain rule

This is the **mathematical foundation** of backpropagation:

- The loss depends on the output layer
- The output layer depends on the previous layer's activations
- Those activations depend on the previous weights
- The chain rule propagates gradients **layer by layer** from output to input

Computation graph

The forward pass can be represented as a **directed acyclic graph (DAG)**:

$$\mathbf{x} \xrightarrow{\mathbf{W}^{(1)}} \mathbf{z}^{(1)} \xrightarrow{\sigma} \mathbf{a}^{(1)} \xrightarrow{\mathbf{W}^{(2)}} \mathbf{z}^{(2)} \xrightarrow{\sigma} \mathbf{a}^{(2)} \xrightarrow{\cdots} \hat{\mathbf{y}} \xrightarrow{\ell} \mathcal{L}$$

Each node is a computation; each edge carries an activation vector.

Backpropagation traverses this graph **right to left** (from $\mathcal{L}$ back to $\mathbf{W}^{(1)}$), applying the chain rule at each node.

Key insight: **intermediate values** computed during the forward pass are **reused** during the backward pass. This is why we store $\mathbf{z}^{(l)}$ and $\mathbf{a}^{(l)}$ during the forward pass.

Error signal — the delta notation

Define the **error signal** (or delta) at layer l as the gradient of the loss with respect to the pre-activation vector $\mathbf{z}^{(l)}$:

$$\boldsymbol{\delta}^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{n_l}$$

This captures: "how much does the loss change when we change the pre-activations of layer l?"

The error signals are computed **recursively** from output to input. Once we have $\boldsymbol{\delta}^{(l)}$, the weight gradients follow immediately:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} \left(\mathbf{a}^{(l-1)} \right)^{\top}$$

Output layer delta: MSE with linear activation

For regression with a **linear** output layer ($\hat{y} = z^{(L)}$, no activation) and MSE loss:

$$\mathcal{L} = \left(\hat{y} - y\right)^2$$

The output layer error signal is:

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial z^{(L)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(L)}} = 2(\hat{y} - y) \cdot 1 = 2(\hat{y} - y)$$

➡️ The error signal is proportional to the **prediction error** $\hat{y} - y$.

Output layer delta: softmax + cross-entropy

For classification with **softmax** output and **cross-entropy** loss, the combined gradient has a remarkably elegant form:

$$\boldsymbol{\delta}^{(L)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(L)}} = \hat{\mathbf{y}} - \mathbf{y}$$

where $\hat{\mathbf{y}}$ is the softmax probability vector and $\mathbf{y}$ is the one-hot target.

This result follows from the chain rule applied to softmax and cross-entropy together.

Output layer delta: softmax + cross-entropy

Properties:

- ✅ Extremely **simple** formula — just the difference between predicted and true probabilities
- ✅ **Numerically stable** — no division by small numbers
- ✅ Directly interpretable: update is proportional to prediction error per class

Backpropagation through hidden layers

Given the error signal $\boldsymbol{\delta}^{(l+1)}$ at layer $l+1$, we compute $\boldsymbol{\delta}^{(l)}$ using the chain rule:

$$\boldsymbol{\delta}^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} = \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \cdot \left(\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \right)^{\top} \cdot \boldsymbol{\delta}^{(l+1)}$$

Backpropagation through hidden layers

Since $\mathbf{z}^{(l+1)} = \mathbf{W}^{(l+1)}\mathbf{a}^{(l)}$ and $\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$, the Jacobian $\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}}$ corresponds to the **weight-only part** of $\mathbf{W}^{(l+1)}$ (the $n_{l+1} \times n_l$ submatrix formed by excluding the bias column, since the bias does not receive an error signal from the previous layer):

$$\boldsymbol{\delta}^{(l)} = \mathbf{W}_{\mathrm{w}}^{(l+1)\top} \boldsymbol{\delta}^{(l+1)} \odot \sigma'\left(\mathbf{z}^{(l)}\right)$$

where $\mathbf{W}_{\mathrm{w}}^{(l+1)} \in \mathbb{R}^{n_{l+1} \times n_l}$ is the weight matrix of layer $l+1$ **excluding the bias column**, and $\odot$ denotes element-wise multiplication.

Convention: $\mathbf{W}_{\mathrm{w}}^{(l)}$ is the submatrix of the full weight matrix $\mathbf{W}^{(l)}$ formed by dropping the last (bias) column. It is used only in the backward pass to propagate error signals between layers.

Weight gradients

Once all error signals $\boldsymbol{\delta}^{(l)}$ are computed, the weight gradients follow from the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} \left(\mathbf{a}^{(l-1)} \right)^{\top}$$

This is an **outer product** between the error signal and the previous layer's activation vector.

Interpretation: weight $W_{jk}^{(l)}$ receives a gradient proportional to:

- How much the loss changes w.r.t. neuron j's pre-activation: $\delta_j^{(l)}$
- How active neuron k in the previous layer was: $a_k^{(l-1)}$

➡️ Weights connecting **active neurons** to **high-error neurons** receive the largest updates.

The backpropagation algorithm

Algorithm: Backpropagation for one training sample $(\mathbf{x}, \mathbf{y})$

1. **Forward pass**: compute and store all $\mathbf{z}^{(l)}, \mathbf{a}^{(l)}$ for $l = 1, \ldots, L$:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)}, \quad \mathbf{a}^{(l)} = \sigma\left(\mathbf{z}^{(l)}\right)$$

2. **Output error signal**: compute $\boldsymbol{\delta}^{(L)}$ from the loss function

3. **Backward pass**: for $l = L - 1, \ldots, 1$:

$$\boldsymbol{\delta}^{(l)} = \mathbf{W}_{\mathrm{w}}^{(l+1)\top} \boldsymbol{\delta}^{(l+1)} \odot \sigma'\left(\mathbf{z}^{(l)}\right)$$

The backpropagation algorithm

4. **Compute weight gradients**: for all layers $l = 1, \ldots, L$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} \left(\mathbf{a}^{(l-1)} \right)^{\top}$$

5. **Update weights**: $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \, \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}$

ReLU activation derivative

For the **ReLU** activation $\sigma(z) = \max(0, z)$, the derivative is:

$$\sigma'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

Applied element-wise in the backward pass:

$$\boldsymbol{\delta}^{(l)} = \mathbf{W}_{\mathrm{w}}^{(l+1)\top} \boldsymbol{\delta}^{(l+1)} \odot \mathbf{1}\left[\mathbf{z}^{(l)} > 0\right]$$

Effect:

- Active neurons ($z > 0$): error signal **passes through unchanged**
- Inactive neurons ($z \leq 0$): error signal is **blocked** (set to zero)

➡️ ReLU backpropagation is computationally cheap — just a binary mask applied to the error signal.

Regression example — setup

Task: predict a target value $y \in \mathbb{R}$ from a single input x

Network: one hidden neuron with ReLU activation, followed by a linear output

$$x \xrightarrow{w^{(1)},\, b^{(1)}} z^{(1)} \xrightarrow{\mathrm{ReLU}} a^{(1)} \xrightarrow{w^{(2)},\, b^{(2)}} \hat{y}$$

Loss: $\ell = (\hat{y} - y)^2$

Regression example — direction of the error signal

Suppose the network **underpredicts**: $\hat{y} < y$

At the output layer, the gradient points in the direction of the prediction error:

$$\frac{\partial \ell}{\partial \hat{y}} = 2(\hat{y} - y) < 0$$

A **negative gradient** means: we should increase $\hat{y}$ to reduce the loss.

➡️ Output weights $w^{(2)}$ and $b^{(2)}$ receive **negative gradients** → they **increase** after the update.

Regression example — error propagation to hidden layer

The error signal propagates backward via the chain rule:

$$\frac{\partial \ell}{\partial a^{(1)}} = \frac{\partial \ell}{\partial \hat{y}} \cdot w^{(2)}$$

If the hidden neuron is active (ReLU output > 0) and $w^{(2)} > 0$:

$$\frac{\partial \ell}{\partial w^{(1)}} = \underbrace{\frac{\partial \ell}{\partial \hat{y}}}_{<0} \cdot \underbrace{w^{(2)}}_{>0} \cdot x$$

For a positive input $x > 0$: the hidden weight also receives a **negative gradient** → $w^{(1)}$ **increases** too.

Classification example — setup

Task: classify an input into one of $K = 3$ classes using a 2-layer MLP

Network: $\mathbf{x} \xrightarrow{\mathbf{W}^{(1)}} \mathbf{z}^{(1)} \xrightarrow{\text{ReLU}} \mathbf{a}^{(1)} \xrightarrow{\mathbf{W}^{(2)}} \mathbf{z}^{(2)} \xrightarrow{\text{softmax}} \hat{\mathbf{y}}$

Scenario:

- True class: **class 1** → one-hot target $\mathbf{y} = [1, 0, 0]^\top$
- Network assigns highest probability to **class 2** instead

$$\hat{\mathbf{y}} \approx \begin{bmatrix} 0.11 \\ 0.79 \\ 0.11 \end{bmatrix}$$

The network is performing **worse than random chance** — a uniform guess over 3 classes gives $1/3 \approx 33\%$ to the true class.

Classification example — output error signal

The combined softmax + cross-entropy gradient has an elegant form:

$$\boldsymbol{\delta}^{(L)} = \hat{\mathbf{y}} - \mathbf{y} = \begin{bmatrix} 0.11 \\ 0.79 \\ 0.11 \end{bmatrix} - \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} -0.89 \\ +0.79 \\ +0.11 \end{bmatrix}$$

For each class k, the sign of $\delta_k^{(L)}$ directly encodes what needs to happen:

- $\delta_1^{(L)} < 0$: score of class 1 is too **low** → must be **increased**
- $\delta_2^{(L)} > 0$: score of class 2 is too **high** → must be **decreased**
- $\delta_3^{(L)} > 0$: score of class 3 is slightly too **high** → must be **decreased slightly**

Classification example — weight updates

Weight gradient for the output layer: $\dfrac{\partial \ell}{\partial \mathbf{W}^{(L)}} = \boldsymbol{\delta}^{(L)} \left(\mathbf{a}^{(L-1)}\right)^{\top}$

Each row of $\mathbf{W}^{(L)}$ corresponds to one class. For every **active hidden neuron** ($a_j^{(L-1)} > 0$), the sign of the update follows $\boldsymbol{\delta}^{(L)}$:

Connection	Gradient direction	Weight update
Hidden neuron → class 1	Negative	**Increases**
Hidden neuron → class 2	Positive	**Decreases**
Hidden neuron → class 3	Positive	**Decreases slightly**

Classification example — weight updates

The same error signal $\boldsymbol{\delta}^{(L)}$ propagates back through $\mathbf{W}^{(L)}$ to update the hidden layer weights.

Key insight: $\boldsymbol{\delta}^{(L)} = \hat{\mathbf{y}} - \mathbf{y}$ directly encodes how much and in which direction each class score must change — making the output gradient both intuitive and numerically stable.

Gradient descent and backpropagation in PyTorch

In practice, you never implement backpropagation by hand. PyTorch's **autograd** engine records every operation performed on tensors and automatically computes all gradients.

Three key concepts:

Concept	What it does
`requires_grad=True`	Tells PyTorch to track this tensor in the computation graph
`loss.backward()`	Traverses the graph backwards and accumulates $\partial \mathcal{L} / \partial p$ in `p.grad` for every parameter p
`optimizer.step()`	Updates each parameter using the stored gradient: $\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} \mathcal{L}$

You define the forward pass. PyTorch handles the backward pass! 😎

PyTorch autograd — minimal example

```python
import torch

# Scalar parameter, tracked by autograd
w = torch.tensor(2.0, requires_grad=True)

# Forward pass: compute a simple loss  L = (w - 3)^2
loss = (w - 3.0) ** 2   # loss = 1.0

# Backward pass: dL/dw = 2*(w-3) = -2
loss.backward()

# Gradient descent step (learning rate = 0.1)
with torch.no_grad():
    w -= 0.1 * w.grad   # w = 2.0 - 0.1*(-2) = 2.2

w.grad.zero_()          # always zero gradients before next step
```

➡️ `w.grad` contains $\partial \mathcal{L} / \partial w$ — no manual derivation needed.

PyTorch training loop — regression

```
import torch
import torch.nn as nn

model = nn.Linear(1, 1) # 1 in, 1 out
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
loss_fn = nn.MSELoss()

for epoch in range(100):
    y_hat = model(x)
    loss = loss_fn(y_hat, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

`optimizer.zero_grad()` must be called **before** `loss.backward()` — otherwise gradients accumulate across steps.

PyTorch training loop — classification with a CNN

```
import torch
import torch.nn as nn

model = MyCNN()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()
for images, labels in dataloader:
    logits = model(images)
    loss = loss_fn(logits, labels)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

➡️ The **same three-line backward-step pattern** works for any architecture — MLP, CNN, RNN, Transformer.

Vanishing gradients

In deep networks, error signals are multiplied through many layers. For a sigmoid activation:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq 0.25$$

After L layers, the gradient is scaled by up to 0.25^L:

- 10 layers: gradient shrinks by factor $\leq 0.25^{10} \approx 10^{-6}$
- Early layers receive nearly **zero gradient** and fail to learn

Solutions:

- ✅ Use **ReLU** activations (derivative = 1 for active neurons)
- ✅ **Batch normalization**: re-centers activations and prevents saturation
- ✅ **Residual connections** (ResNet): add skip connections that bypass layers, providing direct gradient paths

Overfitting and regularization

A network may **memorize** training data and generalize poorly. The training loss decreases but the validation loss increases — this is **overfitting**.

Common regularization techniques:

Technique	Mechanism
L2 regularization (weight decay)	Adds $\lambda \lvert \mathbf{W} \rvert_2^2$ to the loss, penalizes large weights
Dropout	Randomly zeroes activations during training
Data augmentation	Artificially increases training set diversity
Early stopping	Stop training when validation loss starts increasing
Batch normalization	Acts as mild implicit regularization

Overfitting and regularization

L2 regularization modifies the weight update:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta\left(\nabla_{\mathbf{W}}\mathcal{L} + 2\lambda\mathbf{W}\right) = (1 - 2\eta\lambda)\mathbf{W} - \eta\nabla_{\mathbf{W}}\mathcal{L}$$

Gradient Descent & Backpropagation

Recap!

- 🎯 A **loss function** measures prediction error: MSE for regression, cross-entropy for classification.
- 📉 **Gradient descent** iteratively minimizes the loss: $\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} \mathcal{L}$.
- 🔗 **Backpropagation** computes all weight gradients in a single backward pass via the chain rule.
- 💡 Error signals $\boldsymbol{\delta}^{(l)}$ propagate from output to input: $\boldsymbol{\delta}^{(l)} = \mathbf{W}_{\mathrm{w}}^{(l+1)\top} \boldsymbol{\delta}^{(l+1)} \odot \sigma'(\mathbf{z}^{(l)})$.
- 🔢 Weight gradients are outer products: $\partial \mathcal{L} / \partial \mathbf{W}^{(l)} = \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^{\top}$.

Training CNNs

Recap!

- 🔁 CNNs are computation graphs — backpropagation applies to every layer via **autograd**.
- 🤖 In practice, **PyTorch autograd** computes all gradients automatically — you only define the forward pass.
- 🔄 The standard training loop: forward pass → loss → `optimizer.zero_grad()` → `loss.backward()` → `optimizer.step()`.
- 🎯 The same three-line backward-step pattern works for any architecture — MLP, CNN, RNN, Transformer.